

Comparação de Algoritmos para Coloração de Grafos: Guloso vs. Backtracking

Matheus Azevedo de Sá
Rian Carlos Valcanaia

7 de outubro de 2025

1 Abordagem com Algoritmo Guloso

Descrição do Algoritmo

A estratégia gulosa é direta e intuitiva. O algoritmo processa os vértices um a um, em uma ordem pré-definida (neste caso, da ordem 0 a 24). Para cada vértice, ele atribui a menor cor possível que ainda não foi utilizada por nenhum de seus vizinhos já coloridos. Ele nunca reconsidera uma decisão; uma vez que um vértice é colorido, sua cor não é alterada.

Implementação em Python

```
1 def greedy(grafo):
2     num_vertices = len(grafo)
3     cores = [-1] * num_vertices
4     cores[0] = 0
5
6     for i in range(1, num_vertices):
7         cores_indisponiveis = set()
8         for j in range(num_vertices):
9             if graph[i][j] == 1 and cores[j] != -1:
10                 cores_indisponiveis.add(cores[j])
11
12         cor = 0
13         while cor in cores_indisponiveis:
14             cor += 1
15
16         cores[i] = cor
17
18     num_cores_usado = max(cores) + 1
19     resultado_coloracao = {vertice: cor + 1 for vertice, cor in
20                             enumerate(cores)}
21
22     return num_cores_usado, resultado_coloracao
```

Listing 1: Implementação do algoritmo guloso.

Resultados e Análise

A execução do algoritmo guloso no grafo de 25 vértices é praticamente instantânea. O resultado obtido foi uma coloração válida utilizando 10 cores.

Este resultado estabelece um **limite superior** para o número cromático do grafo, ou seja, sabemos que $\chi(G) \leq 10$. No entanto, por ser uma heurística, não há garantia de que 10 seja o menor número de cores possível. A solução ótima pode ser 10 ou um número menor. A vantagem desta abordagem é sua alta velocidade e simplicidade, tornando-a ideal para grafos grandes onde uma solução ótima não é estritamente necessária.

2 Abordagem com Algoritmo de Backtracking

Descrição do Algoritmo

O backtracking é um algoritmo de busca exaustiva que explora sistematicamente o espaço de todas as soluções possíveis. Ele constrói uma "árvore de decisões", onde cada nó representa uma atribuição de cor a um vértice. Se em algum ponto uma atribuição leva a um estado inválido (dois vizinhos com a mesma cor), o algoritmo "poda" todo aquele ramo da árvore e "volta atrás" para tentar uma decisão diferente em um nível anterior.

Para encontrar o número cromático, o algoritmo testa incrementalmente: "É possível colorir com 1 cor? E com 2? E com 3...?". O primeiro número para o qual uma solução completa é encontrada é garantido como o número cromático.

Implementação Otimizada em Python com Heurística de Grau

Para melhorar a eficiência do algoritmo de backtracking em Python, implementamos uma heurística que processa primeiro os vértices com maior grau (ou seja, com mais vizinhos). Essa abordagem tende a reduzir o número de ramos explorados na árvore de backtracking, podando soluções inviáveis mais rapidamente.

```
1 def eh_cor_segura(vertice_atual, matriz_adj, cores_atribuidas,
2   cor_teste):
3     for vizinho in range(len(matriz_adj)):
4       if matriz_adj[vertice_atual][vizinho] and cores_atribuidas[
5   vizinho] == cor_teste:
6       return False
7     return True
8
9 def tenta_colorir_grafo(matriz_adj, num_cores_max, cores_atribuidas,
10  vertices_ordenados, indice_vertice):
11   if indice_vertice == len(vertices_ordenados):
12     return True
13
14   vertice_atual = vertices_ordenados[indice_vertice]
15
16   for cor_teste in range(1, num_cores_max + 1):
17     if eh_cor_segura(vertice_atual, matriz_adj, cores_atribuidas,
18   cor_teste):
19       cores_atribuidas[vertice_atual] = cor_teste
20
21       if tenta_colorir_grafo(matriz_adj, num_cores_max,
22   cores_atribuidas, vertices_ordenados, indice_vertice + 1):
23         return True
```

```

19         cores_atribuidas[vertice_atual] = 0
20     return False
21
22
23 def encontrar_coloracao_minima(matriz_adj):
24     num_vertices = len(matriz_adj)
25     cores_atribuidas = [0] * num_vertices
26
27     # Ordena os vertices pelo grau decrescente
28     graus = [(i, sum(matriz_adj[i])) for i in range(num_vertices)]
29     vertices_ordenados = [v for v, grau in sorted(graus, key=lambda x:
30 x[1], reverse=True)]
31
32     print("Ordem dos vertices por grau (maior -> menor):",
33 vertices_ordenados)
34
35     for num_cores_tentativa in range(1, num_vertices + 1):
36         print(f"Tentando com {num_cores_tentativa} cores...")
37         cores_atribuidas = [0] * num_vertices
38
39         if tenta_colorir_grafo(matriz_adj, num_cores_tentativa,
40 cores_atribuidas, vertices_ordenados, 0):
41             print(f"\nNumero minimo de cores necessario: {
42 num_cores_tentativa}")
43             print("Coloracao resultante (vertice -> cor):")
44             for v in range(num_vertices):
45                 print(f"V{v} -> {cores_atribuidas[v]}")
46             return
47
48     print("\nNao foi possivel colorir o grafo.")

```

Listing 2: Backtracking em Python com ordenação por grau.

Explicação e Adaptações

Inicialmente, o algoritmo de backtracking foi implementado em Python, porém com a complexidade exponencial do problema e pela natureza interpretativa da linguagem, o tempo de execução foi muito alto, não obtendo um resultado após 20 minutos de espera.

Com isso, implementamos o mesmo código em C, para validar a corretude e garantir desempenho, já que nesta linguagem é gerado um executável e o algoritmo é executado após compilação e otimização, obtendo um resultado, mas ainda assim esperando um tempo elevado.

Após isso, partimos para a utilização de uma heurística simples, de ordenação por grau antes de usar o backtracking, já que vértices com maior número de vizinhos são processados primeiro, reduzindo o número de ramos explorados pelo backtracking, permitindo podas antecipadas da árvore de busca. O resultado foi instantâneo, demorando menos de 5 segundos para encontrar o número cromático ótimo.

Resultados e Análise

A execução do algoritmo de backtracking em Python com otimização foi bem-sucedida, confirmando a corretude da implementação e permitindo obter a coloração ótima do grafo. O resultado encontrado foi uma coloração válida utilizando 8 cores, o que determina o número cromático do grafo como $\chi(G) = 8$.

Esse resultado comprova a precisão do algoritmo de backtracking, que busca de forma exaustiva todas as combinações possíveis até encontrar a coloração mínima. Em contrapartida, o tempo de execução foi significativamente maior que o da abordagem gulosa, refletindo sua complexidade exponencial da ordem de $O(m^V)$, onde m é o número de cores e V é o número de vértices.

Embora o algoritmo guloso tenha encontrado uma solução com 10 cores de maneira praticamente instantânea, o backtracking mostrou que o número mínimo necessário é 8. Assim, o método exato valida o limite inferior e evidencia a diferença entre uma solução heurística e a solução ótima.

3 Análise Comparativa

Característica	Algoritmo Guloso	Algoritmo de Backtracking
Velocidade	Extremamente rápido. Complexidade de tempo polinomial (aprox. $O(V^2)$).	Extremamente lento. Complexidade de tempo exponencial ($O(m^V)$). E mesmo com otimização, não garante uma solução rápida para grafos maiores
Optimalidade	Heurístico. Encontra uma solução válida, mas não garante que seja o menor número de cores.	Ótimo. Garante que encontrará o número cromático do grafo.
Uso de Memória	Baixo. Armazena principalmente a matriz e o vetor de cores.	Alto. A pilha de recursão pode crescer até a profundidade V .
Aplicabilidade Prática	Excelente para grafos grandes, onde uma solução "boa o suficiente" e rápida é necessária.	Viável apenas para grafos não muito grandes.

Tabela 1: Tabela comparativa entre as abordagens Gulosa e de Backtracking.

4 Conclusão

A análise dos dois algoritmos evidencia que não há uma "solução única" para o problema da coloração de grafos, mas sim abordagens complementares com diferentes propósitos.

O algoritmo guloso destacou-se por sua capacidade de fornecer uma coloração válida de forma praticamente instantânea, utilizando 10 cores para o grafo de 25 vértices. Essa rapidez e simplicidade fazem dele uma solução prática e eficiente quando o objetivo é obter resultados rápidos e suficientemente bons.

Já o algoritmo de backtracking, apesar de apresentar uma complexidade exponencial e demandar mais tempo de execução, foi capaz de encontrar a **solução ótima** com apenas **8 cores**, determinando o número cromático do grafo como $\chi(G) = 8$. Esse resultado

confirma sua precisão e demonstra o valor de métodos exatos quando a otimalidade é um requisito essencial.

Em síntese, o algoritmo guloso oferece eficiência e escalabilidade, enquanto o backtracking garante precisão e completude. A escolha entre eles depende diretamente das necessidades do problema: quando a velocidade e a viabilidade prática são prioritárias, o método guloso é preferível; quando se busca a solução ótima e o tamanho do grafo permite, o backtracking é a escolha ideal.